

WALMART AI ENGINEER

Complete Interview Roadmap

6 Rounds | All Questions with Expected Answers

Prepared: September 2026

Your Profile	Target Position
2 Years Experience	Senior AI Engineer
9 CTC Current Salary	20-24 CTC Expected
ML Engineer / Data Scientist	Team Lead / IC Level

ROUND 1: PHONE SCREENING

Duration: 30 minutes | Purpose: Verify background, initial communication, problem-solving approach

Q1: "Tell me about yourself and your ML journey. What made you choose this field?"

I'm [Name], an ML engineer with 2 years of hands-on experience building production machine learning systems. I started my career as a data analyst at [Company], working with SQL and Python for business analytics. After 6 months, I transitioned into machine learning because I realized I wanted to solve problems at scale rather than just analyze data.

Over the past 2 years, I've specialized in building end-to-end ML pipelines. My key achievements include: (1) developing a recommendation engine that improved click-through rate by 18% and currently serves 2.5M active users; (2) building a demand forecasting model that reduced inventory costs by \$3M annually with 23% error reduction; and (3) leading the migration of our ML infrastructure to reduce model serving latency by 60%.

I'm drawn to ML because it sits at the intersection of mathematics, engineering, and business impact. I want to join Walmart specifically because your scale—150M users, 50M+ products—presents challenges that most companies never face. The problems you solve in recommendations, pricing optimization, and supply chain forecasting directly impact millions of customers.

Q2: "What is your strongest technical skill and your area of growth?"

My strongest area is building end-to-end ML pipelines using Python, TensorFlow, and scikit-learn. I've deployed 5 production models that currently serve 150K+ predictions per day with 99.5% uptime. I'm particularly strong in feature engineering and model optimization—I can take a raw business problem and deliver a production-grade model within 6-8 weeks.

In terms of growth, I recognize that I haven't worked extensively with distributed computing frameworks like Apache Spark or Hadoop. While I understand the fundamentals and have done small-scale experiments, production-level distributed training is an area I'm actively building. I've already started learning Spark through a personal project and have taken an online course. I know this will be essential at Walmart's scale, and I'm committed to mastering it quickly.

Q3: "Why specifically do you want to join Walmart?"

I want to join Walmart for three specific reasons:

First, unmatched scale: Walmart's data volume and serving requirements are unique. Building systems that handle 100M+ transactions daily requires solving problems that most companies never encounter. This is the level of scale I want to work at.

Second, problem diversity: Unlike companies focused on a single ML problem, Walmart needs optimization across recommendations, pricing, supply chain, demand forecasting, and fraud detection. This breadth will accelerate my growth as an engineer.

Third, measurable business impact: At Walmart, ML decisions directly affect inventory efficiency, customer satisfaction, and operational costs. I want to work on problems where my technical decisions have millions of dollars in business impact. I've researched your recent earnings reports and see that e-commerce optimization is a strategic priority. I'm excited to contribute to that mission.

Q4: "Walk us through your most challenging project. How did you overcome it?"

I led the development of a demand forecasting model for [Company]'s inventory management. The challenges were severe: we had 40% missing values in our time series data due to system outages, simple ARIMA models couldn't capture complex patterns, and we had a strict constraint—predictions had to complete within 30 minutes daily.

My approach: (1) **Data:** I implemented MICE imputation combined with domain knowledge from supply chain experts to handle missing values. (2) **Modeling:** I built a hybrid ensemble—SARIMA for seasonal patterns and LSTM for trend capture—which significantly outperformed single models. (3) **Optimization:** I vectorized computations and used model quantization to reduce inference time.

Results: Achieved 23% MAPE improvement over baseline, reduced forecast error to 7%, and cut inventory carrying costs by \$3M annually. The model was deployed to production and has been running for 18 months with stable performance.

What this taught me: successful ML isn't just about algorithm sophistication—it's about deeply understanding the business problem, being pragmatic about trade-offs, and obsessing over deployment reliability.

ROUND 2: TECHNICAL CODING ASSESSMENT

Duration: 60 minutes | 2 LeetCode-style problems | Tests: DSA, code quality, communication

Q1: Word Ladder II (Graph Problem)

Problem Statement: Given two words (beginWord, endWord) and a dictionary list, return all shortest transformation sequences from beginWord to endWord where each transformation changes exactly one letter. Each transformed word must exist in the dictionary.

Expected Solution Approach:

Time Complexity: $O(N^2 \times L)$ where N = number of words, L = word length

Space Complexity: $O(N^2)$

Key Insights: Use BFS to find shortest path distances (guarantees shortest transformations), then use DFS with distance checking to reconstruct all shortest paths.

Q2: Merge K Sorted Lists (Heap Problem)

Problem Statement: You are given an array of k linked lists, each sorted in ascending order. Merge all linked lists into one sorted list and return it.

Expected Solution Approach:

Time Complexity: $O(N \log k)$ where N = total nodes, k = number of lists

Space Complexity: $O(k)$ for heap

Strategy: Use a min heap to efficiently track the smallest element across all lists. Always extract the minimum and add the next node from the same list. This is optimal compared to $O(N \times k)$ naive approach.

ROUND 3: ML SYSTEM DESIGN

Duration: 90 minutes | Real-world system design problem

Problem: Design a Real-Time Product Recommendation System for Walmart.com

Requirements: 150M monthly active users | 50M+ products catalog | Real-time predictions (<200ms) | A/B testing support | Cold start handling | Cost-effective at scale

PHASE 1: REQUIREMENTS CLARIFICATION (5-10 minutes)

Functional Requirements:

- Recommend top-K products for each user (typically 5-20)
- Real-time serving with latency <200ms
- Handle new users (cold start)
- Handle new products
- Support A/B testing for model variants

Non-Functional Requirements:

- Availability: 99.9%+ uptime
- Scalability: 150M users, millions of daily requests
- Cost: Per-user cost <\$0.0001
- Data freshness: User behavior updated hourly

PHASE 2: SYSTEM ARCHITECTURE (20-30 minutes)

Two-Stage Ranking Pipeline:

Stage 1 - Candidate Generation (1000 → 100):

- Use collaborative filtering to generate initial candidates
- Deploy two-tower neural network architecture (user tower + product tower)
- Use FAISS for fast nearest-neighbor search
- Serve from Redis cache for sub-50ms latency

Stage 2 - Re-ranking (100 → 10):

- Use LambdaMART/XGBoost to score candidates with contextual features
- Features: user preferences, product attributes, interaction history
- Apply business rules (inventory, diversity, freshness)
- Final filtering for business constraints

Serving Infrastructure:

- Load balancer distributes requests across servers
- Redis cache stores pre-computed embeddings (user & product)
- Model serving: TensorFlow Serving or KServe
- Fallback: Return trending products if main system fails

PHASE 3: OFFLINE TRAINING PIPELINE (15-20 minutes)

Data Collection (Daily):

- User events: views, clicks, purchases, cart actions
- Product data: attributes, embeddings, catalog updates
- Context: time, device, location, seasonality

Feature Engineering:

- User features: purchase history, category preferences, price sensitivity
- Product features: popularity, recency, quality score (ratings)
- Interaction features: co-purchase patterns, category affinity
- Contextual: time of day, day of week, seasonality factors

Model Training (Candidate Generation):

- Architecture: $\text{Embedding}(\text{user_id}) \rightarrow \text{Dense}(256 \rightarrow 128) + \text{Embedding}(\text{product_id}) \rightarrow \text{Dense}(256 \rightarrow 128)$
- Loss: Categorical cross-entropy on positive/negative samples
- Optimization: Adam, batch size 1024, learning rate 0.001
- Validation: 80/20 train/test split, evaluate with Recall@100

Model Training (Re-ranking):

- Use gradient boosting (LambdaMART) to learn ranking order
- Requires labeled data: (query, list of products, relevance scores)
- Train on clicks, purchases, user satisfaction signals
- Evaluate with NDCG@10, MAP, Precision@10

PHASE 4: COLD START HANDLING (10 minutes)**New User Cold Start:**

- Return trending products (most popular items)
- Segment by category: show top items in each category
- Use content-based matching if user provides preferences
- Gradually transition to collaborative filtering as data accumulates

New Product Cold Start:

- Use content features (category, attributes) to match with existing products
- Recommend to users who purchased similar products
- Controlled rollout: show new products to 5% of users initially
- Scale up based on performance

PHASE 5: A/B TESTING & MONITORING (15 minutes)**A/B Testing Framework:**

- Traffic split: 90% control (baseline) vs 10% treatment (new model)
- Run for minimum 2 weeks to account for weekly seasonality
- Use t-test for statistical significance ($p < 0.05$)
- Monitor: CTR, conversion rate, average order value, user satisfaction

Metrics to Track (Real-time):

- Click-through rate (CTR)
- Conversion rate (purchases / impressions)
- Revenue per recommendation
- Diversity (% unique categories recommended)
- Latency (p50, p95, p99)

Offline Metrics (Batch evaluation):

- NDCG@10 (normalized discounted cumulative gain)
- Precision@10
- Mean average precision (MAP)
- Catalog coverage (% of products recommended at least once)

ROUND 4: DEEP TECHNICAL INTERVIEW

Duration: 60 minutes | LLMs, Production ML, model drift

Q1: "Explain the Attention Mechanism. Why is it better than RNNs for sequence modeling?"

Attention Mechanism Explanation:

Attention allows each token in a sequence to directly attend to any other token, regardless of distance. The mechanism computes three matrices: Queries (Q), Keys (K), and Values (V). The attention score is computed as: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$

Why Better Than RNNs:

1. **Parallelization:** RNNs process sequences sequentially ($O(n)$ steps), limiting throughput. Attention is fully parallelizable—all tokens can be processed simultaneously.
2. **Long-range dependencies:** RNNs suffer from vanishing gradients when modeling distant dependencies. Attention creates direct paths between all token pairs, making long-range learning easier.
3. **Interpretability:** Attention weights show which parts of the input the model focuses on. RNNs are largely black boxes.
4. **Scalability:** Transformers scale better to larger datasets and longer sequences.

Trade-off: Attention has $O(n^2)$ complexity in sequence length, making it expensive for very long sequences. Solutions: sparse attention, linear attention variants, hierarchical approaches.

Q2: "You built a model with 85% NDCG offline but it only achieves 70% of that performance in production. How do you debug?"

Systematic Debugging Approach:

Step 1: Data Consistency (Priority 1)

Check if training and production data distributions match:

- Plot feature distributions: training vs production
- Use statistical tests (Kolmogorov-Smirnov) to detect drift
- Common issue: Training used last 30 days data, but production serving on last 7 days
- Check for missing features in production pipeline

Step 2: Feature Engineering Pipeline (Priority 1)

Verify features are computed identically offline and online:

- Log actual features used in production predictions
- Compare with training features
- Common issue: Lag features computed differently (off-by-one errors in dates)
- Verify scaling/normalization applied correctly

Step 3: Model Version (Priority 2)

- Confirm correct model version deployed
- Check for precision loss (FP32 vs INT8)
- Verify library versions match (TensorFlow, scikit-learn)

Step 4: Metric Definition (Priority 2)

- NDCG measures ranking quality; CTR depends on position bias
- Item at position 1 gets 80% CTR regardless of relevance
- Solution: Account for position bias in metric evaluation

Step 5: Post-Processing/Filtering (Priority 1)

- Are recommendations filtered after model scores them?
- Business rules removing high-quality predictions?
- A/B test: disable filtering to isolate issue

Action Plan:

1. Instrument production code to log features for 1% of requests
2. Compare offline vs production feature distributions
3. Run A/B test: new model vs baseline (isolates model quality from other factors)
4. If new model still underperforms: issue is likely data or features, not model quality

Q3: "How do you prevent model drift in production? Give specific strategies."**Model Drift Types:**

1. **Data Drift:** Feature distributions change (e.g., user age: avg 35→45 over 6 months)
2. **Concept Drift:** Relationship between features and target changes (e.g., price sensitivity shifts with economy)
3. **Label Drift:** Target distribution changes (e.g., click rate: 10%→5%)

Prevention Strategy 1: Continuous Monitoring

- Track feature distributions daily (mean, percentiles, missing rates)
- Monitor key metrics: CTR, conversion, latency
- Set alerts: CTR drops >5%, feature drift detected ($p < 0.05$)
- Dashboard showing model health: prediction distribution, feature stats

Prevention Strategy 2: Automated Retraining

- Retrain weekly on fresh data (last 7 days)
- If new model outperforms: deploy automatically
- Keep versioned models for quick rollback

Prevention Strategy 3: Shadow Deployment

- Run new model in parallel (serve to 1% of traffic)
- Log predictions from both models
- Compare metrics offline before full rollout
- De-risks large model updates

Prevention Strategy 4: Online Learning

- Update model incrementally with streaming data
- Use SGD to adjust weights based on real-time feedback
- More responsive than batch retraining

Prevention Strategy 5: Model Ensembles

- Combine old and new models: 70% old + 30% new
- Gradually increase new model weight as confidence builds
- Reduces risk of sudden performance degradation

ROUND 5: TAKE-HOME ASSIGNMENT / CASE STUDY

Duration: 24-48 hours | End-to-end ML project

Problem: Build a Demand Forecasting Model for Walmart Inventory Optimization

Dataset: 2 years of daily sales by product | Features: Product attributes, seasonality, promotions, weather, competitor prices

Deliverables: EDA + Model building + Evaluation + Production code + Business recommendations

Expected Solution Structure:

1. EXPLORATORY DATA ANALYSIS (EDA)

- Visualize sales over time: identify trend, seasonality, anomalies
- Analyze by month/day-of-week: Find patterns
- Correlation analysis: Which features matter most?
- Missing data: Imputation strategy
- Key findings to document: Seasonality strength, promotion impact, price elasticity

2. BASELINE MODELS (Essential step)

- Naive forecast: Repeat last value (establishes floor)
- Seasonal naive: Same day last year
- Exponential smoothing with seasonality
- Measure: MAPE, RMSE, MAE

3. FEATURE ENGINEERING

- Temporal: year, month, day, day-of-week
- Lag features: lag_1, lag_7, lag_30, lag_365
- Rolling statistics: 7-day and 30-day mean/std
- External: promotion flag, is_holiday, competitor_price
- Interaction: price $\times$ promotion, seasonality $\times$ promotion

4. ADVANCED MODEL (XGBoost recommended)

- n_estimators=500, max_depth=7, learning_rate=0.05
- Use early stopping on validation set
- Achieve 5-8% MAPE (2x better than baselines)
- Extract feature importance

5. MODEL COMPARISON

- Table: Model | MAPE | RMSE | Training Time
- Visualize: Bar chart comparing MAPE across models
- Business impact: Cost savings from error reduction

6. PRODUCTION CODE

- Save model, scaler, metadata as pickle/JSON
- Create prediction function that handles new dates
- Feature computation logic: reproducible, well-documented
- Error handling and validation

7. BUSINESS RECOMMENDATIONS

- Current performance: stockouts 8%, overstock 12%
- With model: stockouts 2%, overstock 5%

- Estimated savings: \$10-15M annually
- Deployment plan: 100 top-SKUs → pilot → full rollout (6-8 weeks)
- Monitoring approach: daily performance tracking, weekly retraining

ROUND 6: HR & COMPENSATION ROUND

Duration: 45 minutes | Behavioral fit, motivation, negotiation

Q1: "Tell me about yourself—your career journey, what you've accomplished, and why you're interested in Walmart."

I'm a machine learning engineer with two years of production experience building systems that serve millions of users daily. My career began as a data analyst, where I worked with SQL and business intelligence for about six months. I realized my passion was in building scalable ML systems rather than reporting, so I transitioned into ML engineering.

In my current role, I've driven three significant initiatives: First, I developed a collaborative filtering recommendation engine that improved click-through rates by 18% and now serves 2.5 million active users with sub-100ms latency. Second, I built a demand forecasting model using hybrid LSTM-ARIMA ensembles that reduced forecast error by 40%, resulting in \$3 million in annual inventory cost savings. Third, I led the infrastructure migration from batch serving to real-time serving, cutting model latency by 60%.

I'm specifically interested in Walmart because of the scale and complexity of problems you solve. With 150 million users and 50 million products, the challenges in personalization, pricing optimization, and supply chain forecasting exist at a level most companies never encounter. I want to work on problems where ML decisions have measurable impact on operational efficiency and customer experience at that scale. I've researched Walmart's e-commerce growth and believe my technical background in production ML systems aligns perfectly with your strategic priorities.

Q2: "Why are you leaving your current role? What are you looking for in your next position?"

I've had tremendous growth at my current company and I'm grateful for the opportunities. However, I'm at a point where I'm looking for my next growth challenge. Specifically, I'm seeking three things:

First, **greater scale in production systems**. My current platform serves 10 million users. I want to work on systems handling 100+ million concurrent users, which requires solving problems I haven't yet encountered—distributed training, advanced caching strategies, global model serving. Walmart gives me that opportunity.

Second, **increased ownership and leadership responsibility**. Currently, I implement designs created by senior architects. At Walmart, I want to design systems from first principles, lead technical decisions, and mentor junior engineers. I'm looking for a role where I can have end-to-end ownership of product initiatives.

Third, **broader technical exposure**. My expertise is primarily in recommendations and forecasting. Walmart's diverse ML applications—pricing optimization, supply chain, fraud detection, demand sensing—will accelerate my growth across multiple domains. I want to become a generalist ML engineer who can tackle any problem.

This isn't about compensation alone; it's about the next chapter of my career. I'm looking for an organization where I can have a meaningful impact and develop as a technical leader.

Q3: "Tell me about a significant failure or setback. How did you handle it?"

I deployed a collaborative filtering model that showed excellent offline metrics—NDCG of 0.82—but performed poorly in production with a 15% CTR drop compared to baseline. This was a significant setback because the team had invested six weeks in development.

Root cause analysis: Through systematic debugging, I discovered data leakage in my training pipeline. The training data included future user actions (the click label) that weren't available during real-time serving. When deploying, we could only use features available at prediction time, but the model had learned to depend on future information.

Immediate actions: I rebuilt the model using only features available in production—no future information. This took two weeks and resulted in reasonable (though not excellent) performance.

Process improvements: I created a checklist for feature availability: (1) Can this feature be computed in <50ms? (2) Is data always available, or are there gaps? (3) Does this feature use any future information? I also implemented automated validation to detect training-serving skew before models go to production.

Mentoring: I documented this failure and shared the lesson with the team. Three junior engineers now reference my checklist when building features.

This taught me that real-world ML requires obsessing over details I initially overlooked. It's a humbling reminder that excellent offline metrics mean nothing if the model can't execute in production. I now view production constraints as first-class requirements, not afterthoughts.

Q4: "How do you work with non-technical stakeholders? Give me an example."

I built a demand forecasting system for our supply chain team, and this required close collaboration with non-ML stakeholders.

Initial challenge: The supply chain team didn't have ML background. They were skeptical of a 'black box' ML model and wanted to understand how it would improve their planning process.

My approach: (1) **Understand their world:** I attended their planning meetings, learned their forecasting process, and understood why a one-week lead time was critical. I asked questions like, 'What happens if forecast is 10% off?' and 'How do you currently handle seasonal spikes?'

(2) **Translate technical metrics to business impact:** Instead of saying 'MAPE of 7%', I said, 'You'll reduce stockouts from 8% to 2% and cut overstock events from 12% to 5%.' I quantified the savings: '\$3 million annually from reduced carrying costs and emergency orders.'

(3) **Involve them in validation:** I let the supply chain team test the model for two weeks in parallel with their manual forecast. They saw performance before committing. I asked for their feedback and incorporated suggestions.

(4) **Build support systems:** I created a monitoring dashboard they could check daily. I documented a runbook for when to escalate issues. I was available to explain predictions they questioned.

Result: The team not only adopted the model but became advocates. When budget was allocated for the next phase, they requested me by name.

Key learning: Technical excellence alone isn't enough. You must translate your work into stakeholder language and build trust through transparency and collaboration.

Q5: "What are your salary expectations? How do you think about compensation?"

I've researched compensation for this role thoroughly. Based on Levels.fyi data, salary benchmarks for Senior ML Engineers at Walmart with 4+ years of experience range from 20-25 CTC. Accounting for Walmart's scale premium and my specific contributions, I believe a competitive range is 20-24 CTC.

Here's my reasoning: I'm currently at 9 CTC with 2 years of experience. The jump to Walmart represents a 2-3 year career acceleration due to scale and scope. I'm bringing proven experience in production ML systems that have delivered measurable business impact. My demand forecasting work alone saved \$3 million; my recommendation engine improved revenue by millions.

Flexible on structure: I'm open to discussion on how the package is constructed. If base salary has constraints, I'm interested in equity (RSUs over 4 years), performance bonuses (15-20%), or a signing bonus to offset job transition costs. I'd also value relocation support if required.

What matters most: Beyond the number, I'm looking for a role where I have growth potential. Can I grow into a staff/principal engineer role or management track? Does the team invest in skill development? These factors often matter more than base salary.

My target range is 20-24 CTC, ideally structured as Base 16-18 CTC + Bonus 3-4 CTC + Stock 2 CTC annually. But I'm open to creative structures that align with Walmart's compensation philosophy. What's the typical package for this level?

Q6: "Where do you see yourself in 3-5 years?"

In three to five years, I see myself in a senior technical leadership position.

Role progression (IC track): Year 1-2: Senior ML Engineer, owning product initiatives end-to-end and mentoring junior engineers. Year 2-3: Opportunity to move toward a Staff or Principal Engineer role, setting technical direction across teams. This path allows me to build deep expertise while influencing strategy.

Technical growth: I want to deepen my expertise in production ML systems—infrastructure optimization, advanced model architectures, and emerging techniques like diffusion models or retrieval-augmented generation. I also want to contribute to the industry through technical writing or conference talks.

Business impact: I want projects with multi-million dollar impact. At Walmart, this might mean designing a new recommendation system for a product line, or leading the build of a pricing optimization platform. I want decisions I make to directly affect the bottom line.

Mentorship: I want to develop other engineers. I enjoy teaching and seeing junior engineers grow. By year 3-5, I'd like to have mentored 3-4 engineers into senior roles.

At Walmart specifically: I see myself as a go-to technical expert for production ML challenges. Maybe leading a team focused on real-time personalization, or designing ML platforms that other teams use. I want to contribute to Walmart's reputation as a leader in retail AI.

I'm specifically drawn to Walmart for this because you're at a stage where technical leaders can shape culture and strategy. I want to be part of building something meaningful.

SALARY NEGOTIATION & FINAL PREPARATION

SALARY PROGRESSION ROADMAP

Current: 9 CTC (Data Analyst/Junior ML Engineer level)

Target: 20-24 CTC (Senior ML Engineer level)

Justification: 2→4 years experience + level jump + scale increase

Typical Walmart Package Structure:

- Base salary: 60% of total (16-18 CTC)
- Performance bonus: 15-20% annual (3-4 CTC)
- Stock RSUs: 10% annually (2 CTC), vested over 4 years
- Signing bonus: \$50K-\$150K (optional negotiation)
- Relocation support: \$100K-\$150K (if applicable)

Negotiation Tips:

1. Always ask 'What's the initial offer?' before stating expectations
2. If offer is <20 CTC, respond with: 'Based on market research for this level, I was expecting 22-24. What flexibility exists?'
3. Don't accept immediately. Say: 'This is competitive. Let me review the complete package and I'll get back to you by tomorrow.'
4. Negotiate on multiple dimensions: base, bonus, stock, signing, benefits
5. If capped on salary: 'Can we add a performance milestone bonus? If I ship X models with Y impact in 6 months, I get bonus of Z'
6. Never say 'I need the money.' Say: 'Based on market rates and my experience...'
7. Get everything in writing before you resign from current job

Walk-away numbers:

- Minimum: 18 CTC (must be 2x current)
- Comfortable: 20 CTC
- Ideal: 22-24 CTC
- Don't accept below 18 CTC

INTERVIEW DAY CHECKLIST

- ✓ Research: Know Walmart's business model, recent earnings, strategic priorities
- ✓ Prepare: Have 5-7 project stories ready with specific metrics
- ✓ Technical: Solve 50+ LeetCode problems before Round 2
- ✓ System Design: Practice 3-5 ML system design problems
- ✓ Logistics: Test camera/mic/internet 1 day before
- ✓ Sleep: Get 8 hours the night before
- ✓ Dress: Professional attire (video calls = blazer minimum)
- ✓ Arrive early: Join 5-10 minutes before scheduled time
- ✓ Take notes: Write down key interview questions and your answers
- ✓ Ask questions: Prepare 3-5 thoughtful questions about role/team/culture
- ✓ Follow up: Email within 24 hours thanking interviewer and reiterating interest

This roadmap is your comprehensive guide to succeed in every Walmart AI Engineer interview round. Focus on clarity, specific examples with metrics, and genuine enthusiasm for the role. Remember: they want you to succeed. Good luck! ■